

Interactive Theorem Proving: Introduction

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

This Video

- What is interactive theorem proving?
- What are we doing over the next 10 weeks?
- What do I expect you to already know?
- Some Organisational notes

What is ITP'ing?

Formal proofs:

- Recall with me the following Aristotlian syllogism
 - 1: All men are mortal (Assume)
 - 2: Aristotle is a man (Assume)
 - 3: Aristotle is mortal (from 1, 2, and modus-ponens)
- Modus-ponens: if a implies b and a holds, then b holds
- I hope we all agree that this is an unambiguously correct proof

What is ITP'ing?

- Our objective: construct formal proofs
- Unambiguous, correct proofs:
 - Definitions' and theorem statements' meaning and well-formedness
 - Proof correctness
- All of that wrt precisely and concisely stated axioms

What is ITP'ing?

- Proofs and definitions are written in a formal language
- Meaning is defined by set of axioms, e.g.
 - Linear temporal logic axioms
 - More general: Zermelo-Fraenkel set theory:
 - ▶ Extensionality: $\forall s, t. (\forall x. x \in s \leftrightarrow x \in t) \rightarrow s = t$
 - ▶ Regularity: $\forall s. s \neq \emptyset \rightarrow \exists t. t \in s \wedge t \cap s = \emptyset$
 - ▶ Schema: $\forall s, x_1, \dots, x_n. \exists t. \forall y. y \in t \leftrightarrow (y \in s \wedge \phi(s, x_1, \dots, x_n, y))$
 - ▶ ...
 - Other formalisms
 - ▶ Type theory, Peano Arithmetic, First-order logic, and [Higher-order logic \(HOL\)](#)

What is ITP'ing?

- Why formal proofs?
- Initially to “formalise” mathematics
 - E.g. Euclid's elements (300 B.C.) and principia mathematica [Russel and Whitehead 1910]

What is ITP'ing?

- Then in the latter part of the 20th century it was needed for mathematics. Why?
- Proofs became too big to be reviewed by humans:
 - Four-colour theorem [Appel and Haken 1976] (proof contains 1936 case splits)
 - Classification of finite simple groups [100+ authors, 1955-2004] (proof is spread over 500 journal articles, spanning 15k pages)
 - Kepler's conjecture on Sphere packing [Hales 1998] (99% correct, Ann. Math.)

What is ITP'ing?

■ Computer-based formal proofs:

- A computer program checks every step in the proof, wrt precisely stated axioms
- Logical kernel: very simple program performing the checking
- *De-Brujin criterion*: correctness of the proof depends ONLY on the correctness of the logical kernel

What is ITP'ing?

```

1| theory Sqrt2
2| imports Main
3| begin
4| theorem sqrt2_not_rational:
5|   "sqrt (real 2)  $\notin$   $\mathbb{Q}$ "
6| proof
7|   let ?x = "sqrt (real 2)"
8|   assume "?x  $\in$   $\mathbb{Q}$ "
9|   then obtain m n :: nat where
10|     sqrt_rat: "?x = real m / real n" and lowest_terms: "coprime m n"
11|     by (rule Rats_abs_nat_div_natE)
12|   hence "real (m^2) = ?x^2 * real (n^2)" by (auto simp add: power2_eq_square)
13|   hence eq: "m^2 = 2 * n^2" using of_nat_eq_iff power2_eq_square by fastforce
14|   hence "2 dvd m^2" by simp
15|   hence "2 dvd m" by simp
16|   have "2 dvd n" proof
17|     from 2 dvd m obtain k where "m = 2 * k"
18|     with eq have "2 * n^2 = 2^2 * k^2" by simp
19|     hence "2 dvd n^2" by simp
20|     thus "2 dvd n" by simp
21|   qed
22|   with 2 dvd m have "2 dvd gcd m n" by (rule gcd_greatest)
23|   with lowest_terms have "2 dvd 1" by simp
24|   thus False using odd_one by blast
25| qed
26| end
27|
28|

```

What is ITP'ing?

- Computer-based formal proofs in mathematics:
 - Four-colour theorem in Coq [Gonthier 2005]
 - Feit-Thompson theorem in Coq [Gonthier et al. 2012]
 - Kepler's conjecture in Isabelle/HOL and HOL-Light [Hales et al. 2017]
 - Liquid Tensor Experiment in Lean [Scholze, Commelin et al. 2022]
 - Polynomial Freiman-Ruzsa (PFR) conjecture in Lean [Tao et al. 2023]
 - Fermat's Last Theorem project in Lean [Buzzard et al. 2024–present]

What is ITP'ing?

- In CS formal proof is used for *formal verification*:
 - A system and its desired properties are specified in a formal language
- Landmark verified systems:
 - **seL4**: OS Microkernel in Isabelle/HOL [Klein et al. 2009]
 - **CompCert**: compiler for C [Leroy 2009]
 - **CakeML**: compiler for ML in HOL4 [Kumar et al. 2014]
- Industrial adoption (using Isabelle):
 - **AWS Nitro Hypervisor**: Isolation Engine verified in Isabelle
 - **Apple Cryptography**: `corecrypto` verified in Isabelle
 - Intel, Arm, Meta

What is ITP'ing?

■ AI & LLM Revolution in Theorem Proving:

- Foundation models multiply user productivity by nearly an order of magnitude
- Automates tactic prediction, proof search, and lemma generation

■ Synergies (ITPs $\longleftrightarrow$ LLMs):

- **Verifiable Feedback:** Theorem provers eliminate hallucinations by providing rigorous ground-truth RL feedback to train LLMs on mathematics
- **Auto-formalization:** Translating natural language mathematical statements & proofs directly into mechanized formal specifications

■ Key Achievements:

- **IMO Gold/Silver-Medal Level Performance:** AlphaProof & AlphaGeometry (DeepMind)
- Advances in neural theorem provers (DeepSeek-Prover, LeanDojo, Aristotle)

What is ITP'ing?

- Some of my own **research**:
 - Verification of AI, planning algorithms, and software
 - Formalisation of mathematics (esp. complexity theory and combinatorial optimisation)
 - **AI for Mathematics**: Improving mathematical reasoning and ITP capabilities of small, local models
- If that interests any of you for a project / dissertation, let's have a chat!

Course Structure & Learning Outcomes

■ Weeks 1–5: Foundations & AI Integration

- **ITP Foundations (LO1)**: Term syntax, forward/backward reasoning, and structural induction (numbers, lists, trees)
- **Isabelle/HOL Automation (LO1)**: Proof methods, simplifier, sledgehammer, and counterexamples
- **AI Integration with Isabelle (LO4)**: Interfacing Isabelle with LLMs for tactic suggestions and proof search

■ Weeks 6–10: State-of-the-Art Applications

- **Mathematics & Theoretical CS (LO3, LO5)**: State-of-the-art formalisation of algorithms, functional data structures, and discrete mathematics
- **Software Verification (LO2, LO5)**: State-of-the-art programming language semantics, Hoare logic, and verified software (AutoCorres2)

Expected Background Knowledge

- Mathematical induction
- First-order logic and how to formulate statements in it
- Basic functional programming
- I will refresh them throughout the course!

Organisational Points

■ Software & Community:

- We will use the [Isabelle/HOL](#) theorem prover (Cambridge, TUM, contributors)
- Questions: Course Forum + [Isabelle Zulip Community](#)

■ Recommended Books & Reading (Freely Online):

- [Concrete Semantics](#) (Nipkow & Klein)
- [Functional Data Structures and Algorithms](#) (Nipkow, Abdulaziz et al.)
- [Programming and Proving in Isabelle/HOL](#) (Nipkow)

Organisational Points

■ Assessment:

- **50% Coursework:** Formal proof projects & Viva defense
- **50% Exam:** Paper-based written examination

Organisational Points: AI Policy

■ Group Tutorials (Large & Small):

- **Weeks 1–4:**
 - ▶ **Encouraged:** Pasting syntax errors into AI web chat to understand messages
 - ▶ **Please avoid:** Having AI assistants write or edit theory files for you
- **Week 5 onwards:** Broader AI usage welcomed to assist proof development

■ Assessed Coursework:

- The AI policy for coursework will be published with the coursework (Week 5)

■ Crucial Exam Reminder (**50% Paper-Based Exam**):

- Entirely written on paper; mastering proof construction yourself is essential!